Componenti applicazione

Description	Insieme di componenti cooperanti

Un'applicazione Android è composta da diverse componenti che svolgono ruoli specifici nel funzionamento dell'app.

Le componenti principali di un'applicazione Android sono le seguenti:

Attività (Activity)

Un'attività rappresenta una singola schermata dell'applicazione. Ogni schermata visibile dell'app è implementata come un'attività.

Le attività interagiscono tra loro per formare l'esperienza utente complessiva.

Servizio (Service)

Un servizio è un componente che esegue operazioni in background senza fornire un'interfaccia utente. I servizi sono spesso utilizzati per eseguire operazioni a lungo termine o in risposta a eventi anche quando l'app è in background.

Broadcast Receiver

I ricevitori di trasmissione (Broadcast Receiver) gestiscono messaggi del sistema o eventi personalizzati.

Possono essere utilizzati per rispondere a eventi del sistema o a trasmissioni personalizzate inviate da altre parti dell'applicazione o da altre applicazioni.

Content Provider

Un provider di contenuti gestisce l'accesso e la condivisione dei dati tra diverse applicazioni. Fornisce un'interfaccia standard per accedere, aggiornare o modificare i dati all'interno dell'applicazione.

Fragment

Un frammento rappresenta una parte di un'interfaccia utente o un comportamento di un'attività.

I frammenti sono spesso utilizzati per creare interfacce utente più flessibili e riutilizzabili.

Intent

Gli intent sono oggetti utilizzati per avviare attività, servizi, o per inviare e ricevere messaggi di trasmissione. Possono essere utilizzati per avviare componenti e comunicare tra di esse.

Manifesto dell'Applicazione (`AndroidManifest.xml`)

Il manifesto dell'applicazione contiene le informazioni fondamentali sull'app, come i permessi richiesti, le dichiarazioni di componenti e altre configurazioni. È un file XML essenziale per la struttura e la configurazione dell'app.

Risorse

Le risorse includono file XML, immagini, stringhe, colori, ecc., organizzati all'interno della directory `res/`. Sono utilizzate per separare i dati statici dal codice sorgente, facilitando la manutenzione e il supporto per diverse configurazioni.

Risorse di Asset (Assets)

Gli assets contengono file non elaborati come immagini, file audio, HTML, ecc. che possono essere utilizzati direttamente dall'app senza precompilazione.

Una tipica applicazione Android esegue così:

1. Il *Launcher* (che è esso stesso un Activity) avvia la prima Activity dell'App, inviandole un Intent che indica l'intenzione di lanciarla

2. *L'Activity* chiama setContentView() per impostare la sua UI
3. Il *sistema* chiama certi metodi dell'Activity (callback) in risposta alle azioni dell'utente
4. A seconda dei casi, questi metodi potranno
    a. *lanciare altre Activity* (inviando loro opportuni Intent), sia dell'applicazione sia
    di altre applicazioni
    b. *Inviare* Intent ad Activity già in esecuzione o, in broadcast, a tutti gli interessati
    c. *Interagire* con Services in background
    d. *Recuperare* o salvare dati tramite Content Provider
    e. *Terminare l'Activity* (tornando alla precedente)

Avvio di una activity

L'avvio di un'`Activity` in un'applicazione Android è un'operazione comune e può essere eseguito attraverso l'uso degli `Intent`. Un `Intent` rappresenta un'intenzione di eseguire un'azione e può essere utilizzato per avviare un'attività specifica o comunicare con altri componenti dell'applicazione.

Ecco una panoramica del processo di avvio di un'`Activity`:

1. **Creazione di un Intent:**
    - Per avviare un'`Activity`, devi creare un oggetto `Intent`. Puoi specificare il contesto corrente (solitamente l'`Activity` da cui stai avviando l'altra) e la `Class` dell'`Activity` di destinazione.

```
Intent intent = new Intent(this, SecondActivity.class);
```

- Puoi anche aggiungere dati aggiuntivi all'`Intent` per trasmettere informazioni alla `SecondActivity`:

```
intent.putExtra("key", "value");
```

2. **Avvio dell'Activity:**
   - Puoi avviare l'`Activity` chiamando il metodo `startActivity()` e passando l'`Intent` come parametro:

```
startActivity(intent);
```

   - Se desideri ricevere un risultato dalla `SecondActivity`, puoi utilizzare `startActivityForResult()` e implementare il metodo `onActivityResult()` nella `Activity` chiamante.

3. **Ciclo di Vita dell'Activity:**
   - Quando viene avviata un'`Activity`, passa attraverso diverse fasi nel suo ciclo di vita, come `onCreate()`, `onStart()`, `onResume()`. Il sistema Android gestisce automaticamente questi cicli di vita.

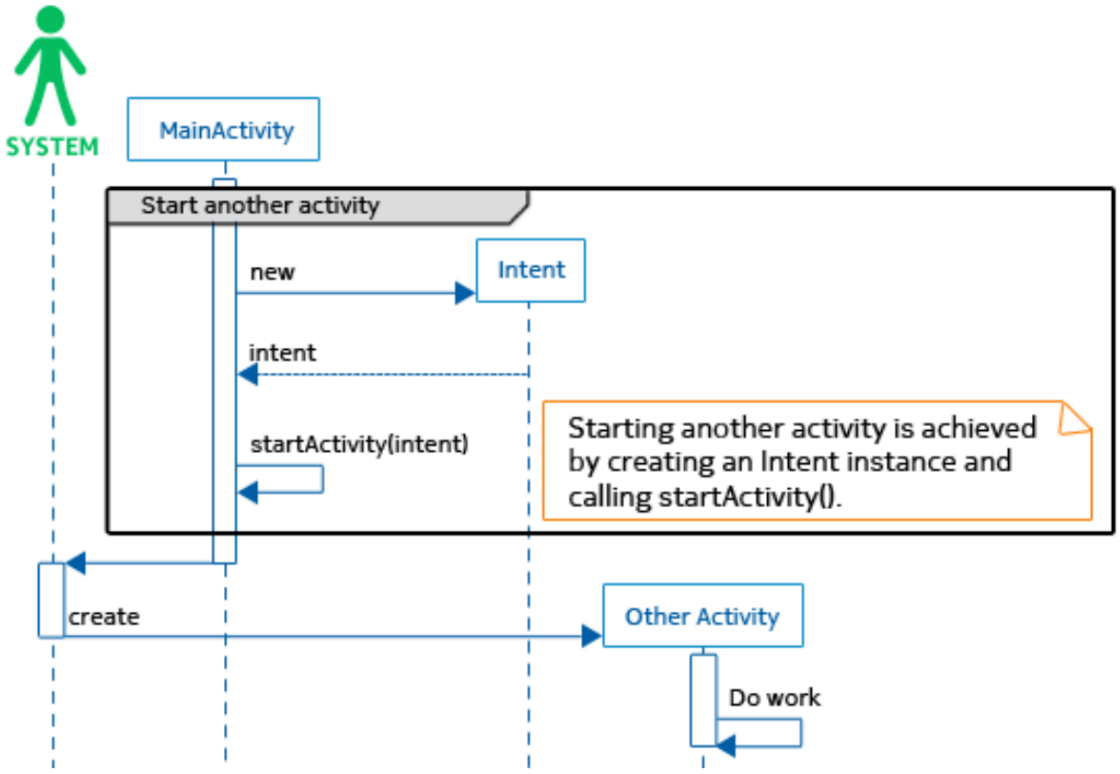
4. **Esecuzione dell'Activity di Destinazione:**
   - La `SecondActivity` viene eseguita e il suo ciclo di vita viene gestito dal sistema Android. Durante questa fase, puoi eseguire tutte le operazioni necessarie, come l'inizializzazione delle risorse, la registrazione di ascoltatori, ecc.

5. **Ritorno alla Activity Chiamante (opzionale):**
   - Se l'`Activity` di destinazione è stata avviata utilizzando `startActivityForResult()`, può restituire un risultato alla `Activity` chiamante. Per fare ciò, la `SecondActivity` imposta un risultato e chiude se stessa:

```
Intent resultIntent = new Intent();
resultIntent.putExtra("resultKey", "resultValue");
setResult(RESULT_OK, resultIntent);
finish();
```

6. **Gestione del Risultato nella Activity Chiamante (opzionale):**
   - Nella `Activity` chiamante, il risultato può essere ricevuto nel metodo `onActivityResult()`:

```
@Override
protected void onActivityResult(int requestCode, int resul
tCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == REQUEST_CODE && resultCode == RESUL
T_OK) {
        String resultValue = data.getStringExtra("resultKe
y");
        // Esegui le operazioni necessarie con il risultat
o ricevuto
    }
}
```



Contenuti di un Intent

1. **Componente di Destinazione:**
   - L'`Intent` specifica quale componente dell'applicazione dovrà essere avviato. Ad esempio, se si intende avviare un'attività, si specifica la classe dell'attività di destinazione:

     ```
     Intent intent = new Intent(context, TargetActivity.class);
     ```

2. **Azione (Action):**
   - L'azione rappresenta l'operazione da eseguire. Può essere un'azione di sistema predefinita o un'azione personalizzata. Ad esempio:

     ```
     intent.setAction(Intent.ACTION_VIEW);
     ```

3. **Dati (Data):**
   - Gli `Intent` possono includere dati che specificano l'oggetto su cui deve essere eseguita l'azione. Ad esempio, se si desidera visualizzare una pagina web, è possibile fornire l'URL come dati:

     ```
     Uri webpage = Uri.parse("https://www.example.com");
     intent.setData(webpage);
     ```

4. **Tipo di Dati (Data Type):**
   - Il tipo di dati indica il tipo di dati associato all'azione. Ad esempio, per un'azione di visualizzazione, il tipo di dati potrebbe essere `text/html` o `image/*`:

     ```
     intent.setType("text/html");
     ```

5. **Parametri Extra (Extras):**
   - Gli `Intent` possono trasportare dati aggiuntivi chiamati "extra". Gli extra sono coppie chiave-valore e possono contenere qualsiasi tipo di dato serializzabile. Ad esempio:

     ```
     intent.putExtra("key", "value");
     ```

6. **Categorie:**
   - Le categorie identificano il tipo di componente o funzionalità che può gestire l'`Intent`. Ad esempio, un'`Intent` può avere la categoria `Intent.CATEGORY_BROWSABLE` se deve essere gestito da un browser.

     ```
     intent.addCategory(Intent.CATEGORY_BROWSABLE);
     ```

7. **Flag:**
   - I flag forniscono indicazioni aggiuntive su come l'`Intent` dovrebbe essere gestito. Ad esempio, `Intent.FLAG_ACTIVITY_NEW_TASK` indica che un'attività deve essere avviata come una nuova attività indipendente.

```
intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
```

8. **Tipo di Riferimento al Componente:**
   - Il tipo di riferimento al componente specifica il modo in cui l'`Intent` dovrebbe essere interpretato. Ad esempio, `Intent.ACTION_DIAL` indica che il componente dovrebbe essere interpretato come un'azione di chiamata telefonica.

     ```
     intent.setAction(Intent.ACTION_DIAL);
     ```

Intent filter

Gli `Intent Filter` in Android sono utilizzati per dichiarare le capacità di un componente (come un'attività, un servizio o un ricevitore di trasmissione) nel file di manifesto (`AndroidManifest.xml`).

Gli `Intent Filter` indicano quali tipi di `Intent` un componente può gestire o rispondere.

Quando un'applicazione o il sistema Android cerca di avviare un componente mediante un `Intent`, il sistema verifica gli `Intent Filter` associati ai componenti nell'applicazione per determinare quale componente è in grado di gestire quell'`Intent`.

Gli `Intent Filter` possono specificare varie informazioni, come azioni, categorie, tipi di dati e altri attributi, per definire le condizioni in cui il componente dovrebbe essere avviato o dichiarato come gestore di un particolare tipo di `Intent`.

Intent filter all'interno di `AndroidManifest.xml`:

```
<activity android:name=".MainActivity">
    <intent-filter>
```

```
        <action android:name="android.intent.action.MAIN"
/>
        <category android:name="android.intent.category.LA
UNCHER" />
    </intent-filter>
</activity>
```

In questo esempio:

- `<action>` specifica l'azione dell'`Intent`, in questo caso, l'azione `MAIN`.
- `<category>` specifica la categoria dell'`Intent`, in questo caso, la categoria `LAUNCHER`.

Quindi, questo `Intent Filter` indica che l'attività `MainActivity` può gestire l'azione `MAIN` e può essere utilizzata come attività di avvio (`LAUNCHER`).

Altri attributi che possono essere specificati negli `Intent Filter` includono:

- `<data>`: Specifica il tipo di dati che l'`Intent` dovrebbe contenere.
- `<scheme>`, `<host>`, `<port>`, `<path>`: Specificano informazioni più dettagliate sulle componenti che possono gestire l'`Intent` in base all'URI.

Gli Intent destinati a componenti diversi vanno lanciati invocando metodi diversi.
Ciascun metodo imposta flag diversi e consegna a destinatari di un certo tipo

In particolare:

- `startActivity(i)` → a un'**Activity**
- `startService(i)`, `bindService(i)` → a un **Service**
- `SendBroadcast(i)` → a un **Broadcast Receiver**
- i Content Provider non usano Intent